

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École supérieure en sciences et technologies de l'informatique et du numérique



Rapport du Mini Projet Collaboratif en Python

Réalisé par

ALKAMA Narima

ALLOU Nadia

BERBACHE Achouak

HEMAIDI Rayene

Environnement Cloud Collaboratif d'Exécution Python

Encadré par

Dr. HASSAOUI Ali

Contents

1	Introduction	3
2	Objectifs du Projet	3
3	Mise en Place des Notebooks Google Colab	3
3.1	Notebook 1 – Fonction moyenne	3
3.2	Notebook 2 – Fonction carré	4
3.3	Notebook 3 – Fonction lire_fichier	4
3.4	Notebook 4 – API _Http	4
4	Debugging Collaboratif	4
4.1	Debugging des erreurs logiques et de typage	5
4.2	Debugging lié à l’environnement d’exécution cloud	5
4.3	Apport du debugging collaboratif	6
5	Versioning Git	6
5.1	Mise en place de la structure de versioning	7
5.2	Versioning dans un environnement cloud (Google Colab)	7
5.3	Gestion des versions successives des notebooks	9
5.4	Différence entre versioning cloud et versioning local	10
5.5	Apport du versioning dans le travail collaboratif	10
6	Documentation – Wiki GitHub	11
7	Environnement et outils de développement	11
8	Conclusion	11
9	Annexe	11

List of Figures

1	Organisation des notebooks dans le dépôt GitHub	7
2	Direct integration of a Google Colab notebook into the GitHub repository using the « <i>Enregistrer dans GitHub</i> » option	8
3	GitHub commit history	9

1 Introduction

Le développement logiciel moderne repose de plus en plus sur des pratiques collaboratives et l'utilisation d'outils cloud. Dans ce contexte, la maîtrise du versioning, du debugging et de la collaboration à distance constitue une compétence essentielle, notamment dans les travaux de recherche et de développement.

Ce mini-projet a pour objectif de mettre en pratique les bonnes pratiques de programmation et de collaboration logicielle en Python, en utilisant des outils cloud et collaboratifs tels que Google Colab et GitHub. Le travail a été réalisé en équipe, conformément aux consignes données lors des ateliers 5.1 et 5.2.

2 Objectifs du Projet

- Mettre en place un environnement Python collaboratif dans le cloud.
- Utiliser le versioning avec Git et GitHub.
- Pratiquer le debugging sur des cas d'erreurs simples.
- Documenter le projet de manière claire et structurée.

3 Mise en Place des Notebooks Google Colab

La création des notebooks Python initiaux a été assurée comme première tâche du projet. Trois notebooks ont été créés et partagés via Google Colab. Chaque notebook contient une fonction simple conçue volontairement pour provoquer une erreur, servant de support aux phases de debugging ultérieures.

3.1 Notebook 1 – Fonction moyenne

Ce notebook contient une fonction moyenne qui calcule la moyenne d'une liste de valeurs. Une erreur se produit lorsque la liste est vide (division par zéro). Ce cas permet d'illustrer la gestion des erreurs et leur correction.

3.2 Notebook 2 – Fonction carré

Ce notebook contient une fonction qui calcule le carré d'une valeur donnée en entrée. Une erreur est provoquée si l'entrée n'est pas numérique, ce qui permet de tester la validation des types et le debugging.

3.3 Notebook 3 – Fonction lire_fichier

Ce notebook contient une fonction `lire_fichier` qui tente de lire le contenu d'un fichier. Une erreur est générée si le fichier n'existe pas, illustrant la gestion des exceptions liées aux fichiers.

3.4 Notebook 4 – API _Http

Ce notebook illustre l'utilisation d'une API REST à l'aide de la bibliothèque `requests`, permettant de récupérer des données au format JSON depuis un service web distant.

La fonction principale, `fetch_posts`, envoie une requête HTTP de type GET vers une URL donnée. Une erreur volontaire est introduite en utilisant une URL incorrecte, ce qui provoque un retour 404 (Not Found) et le déclenchement d'une exception `HTTPError` lors de l'appel à la méthode `raise_for_status()`.

Ce notebook permet ainsi d'aborder la gestion des erreurs réseau et HTTP dans un environnement cloud.

4 Debugging Collaboratif

Dans le cadre de ce mini-projet, une attention particulière a été portée au debugging collaboratif des notebooks Python exécutés dans un environnement cloud, en l'occurrence Google Colab. Cette étape avait pour objectif d'identifier, analyser et corriger les erreurs potentielles liées à la logique du code, au typage des données ainsi qu'à la gestion de l'environnement d'exécution.

Le debugging a été mené de manière systématique selon une démarche en trois étapes : la reproduction de l'erreur, le diagnostic précis de son origine, puis la validation de la correction apportée. Les outils natifs de Python, en particulier le module `pdb`, ont été utilisés afin de permettre une inspection pas à pas de l'état interne des programmes.

4.1 Debugging des erreurs logiques et de typage

Dans certains notebooks, des erreurs pouvaient survenir en raison d'une mauvaise gestion des types de données, notamment lors de l'utilisation de la fonction `input()`, qui retourne systématiquement une chaîne de caractères. Afin d'analyser ce comportement, des cas de test volontairement incorrects ont été introduits.

L'utilisation de points d'arrêt via `pdb.set_trace()` a permis d'inspecter dynamiquement les variables, de vérifier leur type et de comprendre l'origine des exceptions déclenchées. Cette approche a mis en évidence l'importance de la validation des entrées utilisateur et de la conversion explicite des types avant toute opération arithmétique.

Les corrections appliquées incluent :

- la conversion explicite des entrées utilisateur à l'aide de `float(input(...))`,
- l'ajout de contrôles de type à l'aide de la fonction `isinstance`,
- une gestion appropriée des exceptions, notamment `ValueError` et `TypeError`.

4.2 Debugging lié à l'environnement d'exécution cloud

Un autre type d'erreur fréquemment rencontré concerne l'accès aux fichiers dans l'environnement Google Colab. En effet, l'environnement d'exécution étant temporaire, tout fichier externe doit être explicitement créé ou importé avant d'être utilisé.

Lors de l'exécution de fonctions de lecture de fichiers, l'exception `FileNotFoundError` a été reproduite et analysée. Le debugging interactif a permis de vérifier le chemin et le nom du fichier au moment de l'appel, confirmant que l'erreur était liée à l'absence du fichier dans le répertoire courant.

La correction a consisté à :

- intégrer une gestion explicite de l'exception `FileNotFoundError`,
- créer dynamiquement le fichier requis avant sa lecture,
- vérifier le bon fonctionnement après correction à l'aide d'une nouvelle exécution contrôlée.

Dans le même contexte d'exécution cloud, le notebook `API_HTTP` met en évidence des erreurs liées aux communications réseau et aux requêtes HTTP. L'utilisation d'une URL incorrecte lors de l'envoi d'une requête entraîne un code de réponse HTTP 404 (Not Found).

Lorsque la méthode `raise_for_status()` est appelée, cette situation déclenche une exception `HTTPError`, qui a été reproduite et analysée à l'aide du module `pdb`. Le debugging interactif a permis d'inspecter l'URL, le code de statut HTTP et la réponse du serveur, facilitant ainsi l'identification de la cause de l'erreur et la validation de la correction apportée.

4.3 Apport du debugging collaboratif

Le debugging réalisé dans ce projet ne s'est pas limité à la correction ponctuelle d'erreurs, mais a contribué à améliorer la robustesse du code, à renforcer la compréhension du comportement des programmes et à assurer une exécution reproductible dans un environnement cloud partagé.

Il a également facilité la collaboration entre les membres de l'équipe grâce à l'utilisation de notebooks versionnés et documentés sur GitHub. Chaque notebook débogué inclut ainsi une version fonctionnelle finale, des cas de test illustrant les erreurs possibles, une cellule de debugging optionnelle clairement identifiée, ainsi que des commentaires expliquant les causes des erreurs et les corrections apportées.

Le debugging collaboratif mis en œuvre dans ce mini-projet a permis d'assurer la fiabilité des notebooks Python tout en respectant les contraintes d'un environnement cloud. L'utilisation combinée de Google Colab, GitHub et des outils de debugging Python constitue une approche efficace pour le développement, la maintenance et l'analyse de scripts Python en contexte collaboratif.

5 Versioning Git

La gestion des versions, également appelée *versioning*, correspond à l'ensemble des pratiques et des outils permettant de suivre, enregistrer et organiser les différentes évolutions d'un projet logiciel au cours du temps. Elle vise à assurer la traçabilité des modifications apportées au code, la conservation de l'historique des versions successives, ainsi que la possibilité de revenir à un état antérieur en cas d'erreur ou de régression.

Dans un contexte de développement collaboratif, le versioning joue un rôle fondamental en facilitant la coordination entre les membres de l'équipe, en limitant les conflits de modifications et en garantissant la cohérence globale du projet.

Dans ce mini-projet, le versioning a été mis en place à l'aide de Git et de la plateforme

GitHub, en combinaison avec l'environnement cloud Google Colab. Cette approche a permis de suivre de manière structurée l'évolution des notebooks Python, d'archiver chaque modification significative sous forme de commits et d'assurer une collaboration efficace et contrôlée tout au long du projet.

5.1 Mise en place de la structure de versioning

Dès le début du projet, un répertoire dédié, `notebooks/`, a été ajouté au dépôt GitHub de ce mini-projet afin de centraliser l'ensemble des notebooks Python du projet. Afin de garantir le suivi de ce dossier par Git, même lorsqu'il était initialement vide, un fichier `.gitkeep` a été utilisé.

Chaque notebook a été ajouté progressivement au dépôt, en respectant une organisation claire et cohérente. Les fichiers ont été renommés de manière explicite afin de refléter leur fonctionnalité, tels que `notebook_moyenne.ipynb`, `notebook_carre.ipynb` et `notebook_lire_fichier.ipynb`.

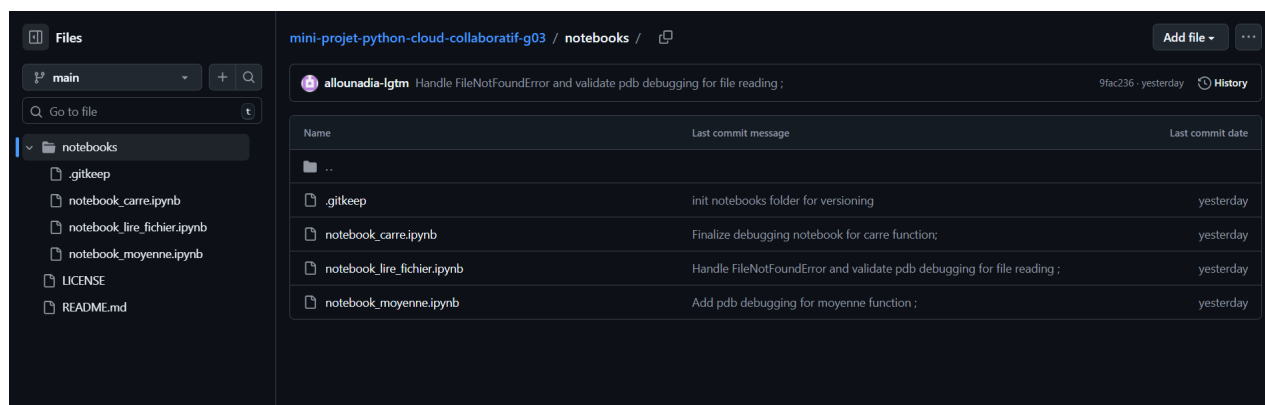


Figure 1: Organisation des notebooks dans le dépôt GitHub

5.2 Versioning dans un environnement cloud (Google Colab)

La majorité du développement a été réalisée dans un environnement cloud à l'aide de Google Colab, choisi pour sa facilité d'accès et ses capacités de collaboration. Le processus de versioning mis en place s'est articulé autour de plusieurs étapes successives et bien définies :

1. **Création des notebooks dans Google Colab** : Les notebooks Python ont été créés et exécutés directement dans Google Colab, permettant de développer et de tester le code dans un environnement cloud partagé.

2. **Intégration des notebooks dans le dépôt GitHub** : Une fois une première version fonctionnelle obtenue, les notebooks ont été enregistrés dans le dépôt GitHub à l'aide de l'option « *Enregistrer dans GitHub* ». Cette étape correspond à la création de la version initiale du notebook dans le système de contrôle de versions.



Figure 2: Direct integration of a Google Colab notebook into the GitHub repository using the « *Enregistrer dans GitHub* » option

3. **Gestion des versions successives** : Après l'intégration initiale, chaque modification significative du code (correction d'erreurs, amélioration de la logique, ajout de contrôles ou de gestion des exceptions) a donné lieu à un nouveau commit Git, accompagné d'un message descriptif.
4. **Suivi de l'historique et traçabilité des modifications** : GitHub a permis de conserver un historique détaillé des versions successives, offrant la possibilité de consulter les anciennes versions, de comparer les évolutions et de revenir à un état antérieur si nécessaire

Cette démarche permet d'assurer un versioning efficace sans recourir aux commandes Git locales, tout en garantissant une traçabilité complète et une gestion rigoureuse de l'évolution des notebooks Python.

Handle FileNotFoundError and validate pdb debugging for file reading ;	9fac236			
allounadia-igtm committed yesterday				
Finalize debugging notebook for carre function; ^{new}	515068d			
allounadia-igtm committed yesterday				
Add pdb debugging for moyenne function ; ^{new}	5190f96			
allounadia-igtm committed yesterday				
handle missing file in lire_fichier (v2)	a6c4b00			
hemadi-cloud committed yesterday				
fix input type for carre (v2)	f077c83			
hemadi-cloud committed yesterday				
add lire_fichier notebook (v1)	5f5a337			
hemadi-cloud committed yesterday				
add carre notebook (v1)	44eca3f			
hemadi-cloud committed yesterday				
fix moyenne empty list (v1)	50ec528			
hemadi-cloud committed yesterday				
moyenne original version (v0)	05f5977			
hemadi-cloud committed yesterday				
Crée à l'aide de Colab	4e32400			
hemadi-cloud committed yesterday				

5.4 Différence entre versioning cloud et versioning local

Il est essentiel de distinguer le processus de versioning réalisé dans un environnement cloud de celui effectué dans un environnement de développement local. En local (par exemple à l'aide d'un éditeur comme VS Code), la gestion des versions repose sur l'utilisation explicite des commandes Git exécutées en ligne de commande, telles que :

```
git add notebook.ipynb
git commit -m "message"
git push
```

Ces commandes permettent respectivement d'indexer les fichiers modifiés, d'enregistrer les changements sous forme de commit et de synchroniser le dépôt local avec le dépôt distant sur GitHub.

À l'inverse, dans un environnement cloud tel que Google Colab, ces étapes sont abstraites et intégrées dans une interface graphique unique. L'utilisateur peut enregistrer directement le notebook dans le dépôt GitHub sans manipuler explicitement les commandes Git. Malgré cette différence de workflow, le résultat demeure strictement équivalent : chaque modification significative est enregistrée sous forme de commit dans le dépôt GitHub, garantissant ainsi la traçabilité, la cohérence et l'historique complet du projet.

5.5 Apport du versioning dans le travail collaboratif

Le versioning a constitué un élément clé dans la réussite du travail collaboratif au sein de l'équipe. Il a permis :

- de conserver un historique exhaustif des modifications apportées aux notebooks ;
- de restaurer rapidement une version antérieure en cas d'erreur ou de régression ;
- de coordonner efficacement les contributions des différents membres du projet ;
- d'assurer une intégration progressive, contrôlée et cohérente des corrections et améliorations.

En conclusion, la mise en place d'un système de versioning rigoureux, basé sur Git et GitHub, a permis de garantir la fiabilité, la traçabilité et la cohérence globale du projet. Cette

approche respecte pleinement les bonnes pratiques du développement logiciel collaboratif, en particulier dans un environnement cloud, et a contribué à structurer efficacement le travail d'équipe tout au long du mini-projet.

6 Documentation – Wiki GitHub

La documentation du projet a été réalisée à l'aide du Wiki GitHub. Les pages du Wiki expliquent les objectifs du projet, l'utilisation des notebooks, le debugging, et le versioning.

7 Environnement et outils de développement

Le mini-projet a été réalisé en utilisant le langage Python comme base de développement. L'environnement cloud Google Colab a permis la création et l'exécution des notebooks, tandis que GitHub a été utilisé pour le partage du code et le suivi des différentes versions. Le module pdb a été employé pour effectuer le debugging interactif et analyser le comportement du programme lors de l'exécution.

8 Conclusion

Le travail réalisé a permis de développer un ensemble de notebooks fonctionnels dans un environnement cloud. Les erreurs rencontrées ont été identifiées, analysées et corrigées de manière progressive. L'utilisation des outils collaboratifs a facilité l'organisation du travail et le suivi des modifications.

9 Annexe

[Lien Github](#)